

Documentando a API com Swagger/OpenAPI

Aprenda a transformar sua API NestJS em uma fonte oficial de consulta interativa, com documentação automática, exemplos de requisição e suporte a autenticação JWT.

Objetivos de Aprendizagem

Ao final deste material, você será capaz de:

1

Compreender a importância

Entender por que documentar APIs REST é essencial para equipes e integrações.

2

Configurar o Swagger

Instalar e configurar o Swagger em uma aplicação NestJS do zero.

3

Documentar endpoints e DTOs

Descrever controladores, operações, respostas e estruturas de dados.

4

Autenticação JWT

Documentar e testar rotas protegidas com Bearer Token diretamente pelo Swagger.

Por que Documentar uma API?

Uma API pode funcionar perfeitamente e ainda assim ser difícil de utilizar. Quando outro desenvolvedor precisa consumi-la, surgem perguntas inevitáveis:

Perguntas sem documentação

- Quais endpoints existem?
- Quais parâmetros devem ser enviados?
- Quais dados são retornados?
- Quais erros podem ocorrer?
- Como a autenticação deve ser realizada?

Com Swagger/OpenAPI você obtém

- Redução de dúvidas da equipe
- Facilidade de manutenção contínua
- Integração mais rápida com o frontend
- Onboarding simplificado para novos membros
- Testes rápidos diretamente pelo navegador

A própria API gera a documentação interativa automaticamente.

O que é OpenAPI?

OpenAPI é uma especificação utilizada para descrever APIs REST de forma padronizada. O Swagger é a ferramenta que interpreta essa especificação e gera uma interface visual interativa.



Rotas e Métodos HTTP

Define todos os endpoints disponíveis e os métodos aceitos: GET, POST, PUT, DELETE.



Parâmetros

Descreve os parâmetros de rota, query string e corpo da requisição.



Estruturas de Dados

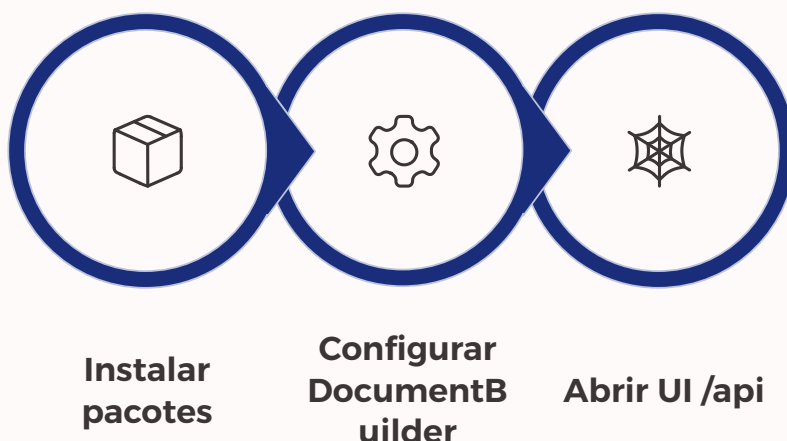
Documenta os schemas dos objetos enviados e recebidos pela API.



Autenticação

Especifica as regras de segurança, incluindo Bearer Token e outros esquemas.

Instalando e Configurando o Swagger no NestJS



Siga os passos abaixo para ter o Swagger funcionando em sua aplicação NestJS.

Passo 1 – Instalar os pacotes

```
pnpm install @nestjs/swagger swagger-ui-express
```

Passo 2 – Importar no main.ts

```
import { SwaggerModule, DocumentBuilder } from '@nestjs/swagger';
```

Passo 3 – Configurar o DocumentBuilder

```
const config = new DocumentBuilder()
  .setTitle('Users API')
  .setDescription('API para gerenciamento de usuários')
  .setVersion('1.0')
  .build();

const document = SwaggerModule.createDocument(app, config);
SwaggerModule.setup('api', app, document);
```

Após iniciar com `pnpm run start:dev`, acesse `http://localhost:3000/api` para visualizar a interface Swagger.

Organizando e Documentando Controladores

Com o Swagger configurado, o próximo passo é organizar os endpoints por grupos e adicionar descrições significativas a cada operação.

@ApiTags – Agrupando endpoints

Agrupar todos os endpoints de um controlador em uma seção nomeada na interface Swagger.

```
@ApiTags('Users')
@Controller('users')
export class UsersController {}
```

@ApiOperation – Descrevendo operações

Adiciona um resumo legível para cada endpoint, facilitando a compreensão do seu propósito.

```
@ApiOperation({ summary: 'Cadastrar usuário' })
@Post()
create() {}
```

@ApiResponse – Documentando respostas

Informa os possíveis retornos HTTP de cada endpoint, incluindo sucesso e erros esperados.

```
@ApiResponse({ status: 201, description: 'Usuário criado com sucesso' })
@ApiResponse({ status: 409, description: 'E-mail já cadastrado' })
```

Documentando DTOs com @ApiProperty

Por padrão, o Swagger não conhece o significado dos campos de um DTO. O decorator `@ApiProperty` resolve isso, expondo exemplos e metadados diretamente na interface.

Exemplo de DTO documentado

```
import { ApiProperty } from '@nestjs/swagger';

export class CreateUserDto {

  @ApiProperty({
    example: 'Maria Silva',
    required: true
  })
  name: string;

  @ApiProperty({
    example: 'maria@email.com',
    required: true
  })
  email: string;

  @ApiProperty({
    example: '123456',
    required: true
  })
  password: string;
}
```

Campos opcionais

Para campos que não são obrigatórios, utilize `required: false`:

```
@ApiProperty({
  example: 'Descrição opcional',
  required: false
})
```

```
})  
bio?: string;
```

Com `@ApiProperty` configurado, o Swagger exibirá automaticamente exemplos de requisição no painel interativo, facilitando os testes.

Resultado na interface

- Campos listados com seus tipos
- Exemplos preenchidos automaticamente
- Indicação visual de campos obrigatórios

Documentando Autenticação JWT

Para APIs que utilizam autenticação baseada em JWT, o Swagger permite configurar o esquema Bearer Token e testar rotas protegidas diretamente pela interface.

1

1. Adicionar ao DocumentBuilder

```
const config = new DocumentBuilder()  
  .setTitle('Users API')  
  .setDescription('API de usuários')  
  .setVersion('1.0')  
  .addBearerAuth()  
  .build();
```

2

2. Proteger o controlador

```
import { ApiBearerAuth } from '@nestjs/swagger';  
  
@ApiBearerAuth()  
@Get()  
findAll() {}
```

3

3. Autorizar no Swagger UI

Após o login, copie o `access_token` retornado, clique em **Authorize** e informe `Bearer seu_token`. O Swagger enviará o token automaticamente.

Exemplo Completo de Endpoint Documentado

Veja como um endpoint totalmente documentado se parece na prática, combinando todos os decorators aprendidos:

```
@ApiTags('Users')
@Controller('users')
export class UsersController {

    @Post()
    @ApiOperation({
        summary: 'Cadastrar usuário'
    })
    @ApiResponse({
        status: 201,
        description: 'Usuário criado com sucesso'
    })
    @ApiResponse({
        status: 409,
        description: 'E-mail já cadastrado'
    })
    create(
        @Body() dto: CreateUserDto
    ) {
        return this.userService.create(dto);
    }
}
```

@ApiTags

Agrupa o controller

@ApiOperation

Descreve a operação

@ApiResponse

Documenta retornos

@Body() DTO

Expõe o schema

Checklist de Validação e Conclusão

Antes de concluir este material, verifique se todos os itens foram implementados corretamente em sua aplicação:

Checklist de Implementação

- ✓ Swagger instalado com pnpm
- ✓ Swagger configurado no main.ts
- ✓ Endpoint /api funcionando no navegador
- ✓ Controllers agrupados com @ApiTags
- ✓ Endpoints documentados com @ApiOperation
- ✓ Respostas documentadas com @ApiResponse
- ✓ DTOs documentados com @ApiProperty
- ✓ JWT configurado com addBearerAuth()
- ✓ Rotas protegidas com @ApiBearerAuth
- ✓ Testes realizados diretamente pelo Swagger

Acessando a Interface Swagger

Após configurar o Swagger em sua aplicação NestJS, você pode acessar a interface interativa no navegador. Certifique-se de que a aplicação está em execução.

Para isso, execute `pnpm run start:dev` no terminal.

Uma vez que a aplicação esteja rodando, a interface Swagger UI estará disponível em <http://localhost:3000/api>.

1. Inicie a aplicação

Execute `pnpm run start:dev` no terminal para iniciar o servidor NestJS.

2. Abra o navegador

Navegue até <http://localhost:3000/api/docs> em seu navegador preferido.

3. Explore os endpoints

Todos os endpoints documentados aparecerão agrupados por `@ApiTags`.

4. Teste uma requisição

Expanda um endpoint, clique em "Try it out", preencha os campos necessários e clique em "Execute".

5. Veja a resposta

O corpo da resposta, o código de status HTTP e os cabeçalhos serão exibidos abaixo da área de teste.

 A rota `/api` pode ser personalizada no `SwaggerModule.setup()`. Por exemplo, `SwaggerModule.setup('docs', app, document)` disponibilizaria a interface em `/docs`.

Além da interface visual, a especificação OpenAPI em formato JSON para integração com outras ferramentas está disponível em <http://localhost:3000/api-json>.

Conclusão

Uma API profissional não deve depender da leitura do código para ser utilizada. A documentação gerada pelo Swagger transforma a própria aplicação em uma fonte oficial de consulta.

Desenvolvedores entendem rapidamente como consumir cada endpoint, quais dados devem ser enviados e quais respostas podem ser esperadas.

A partir deste momento, sua API passa a oferecer não apenas funcionalidades, mas também uma experiência de integração muito mais amigável e produtiva.

